

aict

A local-first workspace for EU AI Act

first-pass compliance triage

Cited. Multi-agent. Runs on your laptop.

The problem

- The EU AI Act is in force, and product teams are the first line of triage.
- Today that triage looks like a shared doc, a Slack thread, and a guess.
- Legal review is expensive — teams need a structured **first pass** before they escalate.
- LLMs alone hallucinate citations. For compliance work, that is worse than nothing.

"Is this use case in scope? Is it high-risk? What do we owe? What's missing?"

Teams need a defensible draft, fast.

What aict does

For one AI use case, on a developer laptop:

1. Create a local case and describe the use case.
2. Upload supporting documents (product brief, vendor note, governance note, DPIAs...).
3. Run a **first-pass EU AI Act assessment** with named agents.
4. Every claim is **cited to a stored chunk** — uploaded doc or AI Act reference.
5. Ask follow-up questions against the saved case context.

Output is always labeled:

This is a decision-support draft, not final legal advice.

MVP scope — intentionally narrow

In scope

- One local case at a time, multiple uploaded documents.
- Local parsing, chunking, retrieval over a bundled AI Act reference corpus.
- Multi-agent analysis with named-agent trace.
- Deterministic citation verification before results are saved.
- Report timeline (history of saved snapshots) and follow-up chat.

Out of scope (on purpose)

- Auth, multi-tenant, cloud hosting, payments.
- Jurisdiction-by-jurisdiction legal conclusions.
- Approvals, redlining, document generation.

Hackathon discipline: ship a credible vertical slice, not a half-built platform.

Demo: HR candidate-screening assistant

An HR team wants AI to summarize CVs, score applicants, and rank candidates in the EU.

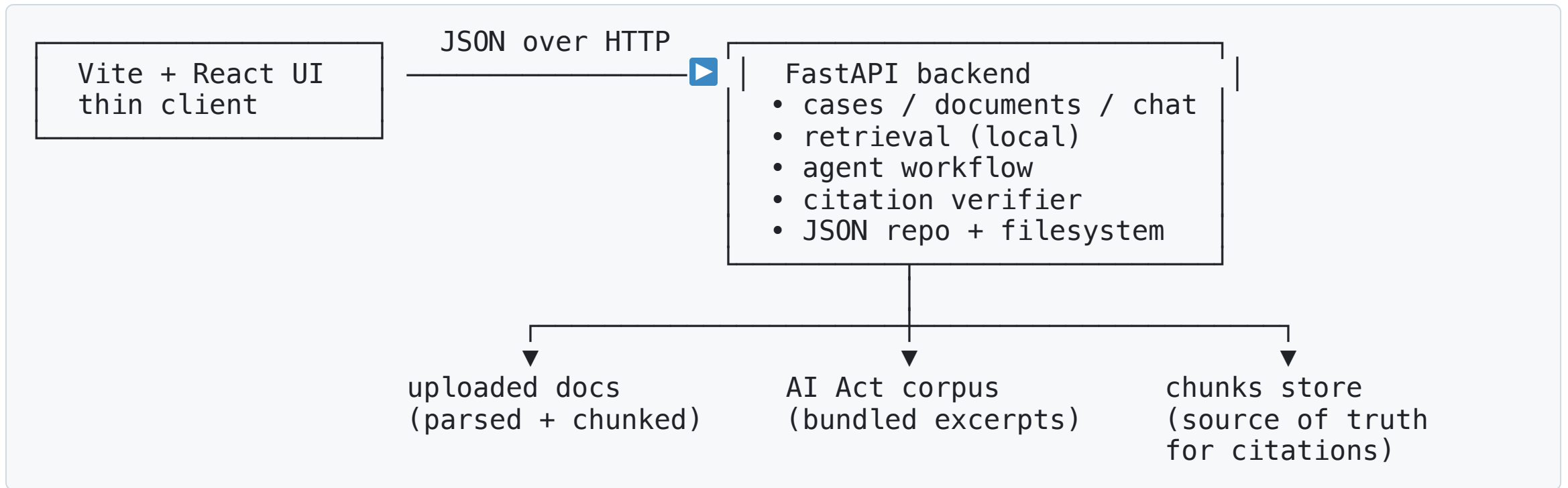
Three uploaded documents drive the demo:

- `demo-hr-product-brief.md` — what the system does and where.
- `demo-hr-vendor-note.md` — model inputs, training data, oversight, logging.
- `demo-hr-governance-note.md` — current process, missing bias testing, planned DPIA.

Expected first-pass result

- Likely **in scope** as an AI system.
- Likely **high-risk** — affects employment / worker management.
- **Uncertainty flagged** where the docs don't prove provider role, validation, bias controls, or post-market monitoring.

Architecture at a glance



Docker Compose starts both containers with a persistent `backend-data` volume.

Multi-agent workflow

A stateful orchestrator fills a shared `AnalysisResult` draft:

Agent	Job
DocumentFactAgent	Extracts facts from uploads, marks missing/uncertain ones.
AISystemDefinitionAgent	Is this an AI system in scope of the Act?
RiskClassificationAgent	Prohibited / high / limited / minimal / unclear — with reasoning.
ObligationsGovernanceAgent	Likely roles, obligations, transparency/GPAI relevance.
CriticUncertaintyAgent	Catches unsupported claims, contradictions, overconfidence.

Each agent attaches **citations to retrieved chunks**. The audience sees the named trace in the UI — not a black box.

Citations are not free-form text

The hard part of LLM compliance work is **provenance**.

aict treats the local **chunks** store as the source of truth:

1. Agents retrieve chunks (uploaded docs **and** bundled AI Act excerpts).
2. Agents may attach citations referencing those chunks.
3. The **deterministic CitationVerifier** runs before the report is saved:
 - The cited chunk ID must exist.
 - The cited snippet must be grounded in the stored chunk text.
4. Unverified citations are stripped or flagged — they never appear as supported claims.

If an agent can't cite, it must mark the claim as an **assumption** or **uncertainty**.

Uncertainty is a feature, not a bug

Most LLM tools optimize for sounding confident. Compliance work needs the opposite.

aict surfaces:

- **Missing information** — what the uploads don't prove.
- **Follow-up questions** — what to collect before a human review.
- **Confidence labels** per assessment section.
- **Limitation notice** on every saved report.

Follow-up chat example from the demo:

"Can we say this is definitely compliant if a recruiter reviews every recommendation?"

The assistant declines a final legal conclusion, explains remaining uncertainty, and cites available sources.

Tech stack & local-first choices

Backend — Python · FastAPI · Pydantic · pypdf · httpx · OpenAI/Anthropic SDKs · pytest

Frontend — React · TypeScript · Vite · Tailwind utilities · lucide-react

Storage — local JSON files (`backend/data/state/...`) with atomic writes; uploads + extracted text on disk

Retrieval — local chunk store, lexical / lightweight vector search; embeddings local by default

LLMs — per-agent model config (OpenAI / Anthropic / vLLM / Ollama-compatible); **no silent mock mode** — missing keys return `llm_not_configured`

Runtime — `make up` → Docker Compose: frontend on `:5173` , backend on `:8001` , persistent demo volume

Why this approach

- **Local-first** — sensitive case material never leaves the laptop unless the user chooses a cloud model.
- **Cited or it didn't happen** — deterministic verification means the audience trusts the report.
- **Named agents, visible trace** — reviewers can see *why* a conclusion was drawn, not just *what*.
- **Uncertainty over false confidence** — the product is decision support, and it says so.
- **Small surface, finishable in a hackathon** — one case, one report at a time, follow-up chat. No yak-shaving.

What's next

- Broader AI Act corpus coverage and structured cross-references.
- Pluggable retrieval (BM25 + embeddings hybrid, configurable per case).
- Multi-case workspaces and shareable read-only report links.
- Evaluation harness over a labeled set of public AI use cases.

Try it

```
make up          # http://127.0.0.1:5173  
make health      # {"status":"ok"}
```

Thank you — questions?